Mini Java Compiler - Phase 1 Technical Report: Lexical Analyzer

Course: CSCI415 - Compiler Design (Fall 2026)

Deliverable: Phase 1 Official Submission Report (Lexical Analysis)

Target Language: Mini Java

Implementation: Python (Standard Library Only, No External Packages)

Team 9 - Members and Roles

- **Member 1:** Hamza Taher Essam (ID: 231001880) - Foundations, Driver, and CLI Integration
- **Member 2:** Mostafa Abd Elhamied Ismael (ID: 231000842) - Central Lexer Engine and Position Tracking
- **Member 3:** Yousef Mahmoud Ali (ID: 231001086) - Team Leader, Symbol Table, Test Samples, and Report
- **Member 4:** Yosef Mohamed Yacoup (ID: 231000016) - Identifier and Number Scanner, PDF Acceptance Suite
- **Member 5:** Youssef Hassan Nasr (ID: 231000751) - String and Operator Scanner, Golden Test Suite

---

1. Tokens Implemented

The lexical analyzer scans source code character-by-character without regular expressions or third-party libraries. All token classifications match the course specifications:

1.1 Keywords (15 Total)

- Control Flow: `if` (`IF`), `else` (`ELSE`), `while` (`WHILE`), `for` (`FOR`), `return` (`RETURN`)
- Primitive Types: `boolean` (`BOOLEAN_TYPE`), `int` (`INT_TYPE`), `float` (`FLOAT_TYPE`), `char` (`CHAR`), `String` (`STRING_TYPE`), `void` (`VOID`)
- Object-Oriented: `class` (`CLASS`), `public` (`PUBLIC`), `static` (`STATIC`), `main` (`MAIN`)
- Keyword matching is strictly case-sensitive (`class` is a keyword; `Class` is scanned as an identifier).

1.2 Literals

- **Numeric Literals (`NUMBER`):** Integers (`10`, `0`) and decimals (`85.5`, `3.14`). Floating-point numbers require digits on both sides of the decimal point.
- **Boolean Literals (`BOOLEAN`):** `true` and `false`.
- **String Literals (`STRING`):** Text enclosed in single (`'A'`) or double (`"Ali"`) quotes. Quotes are preserved in the token lexeme.

1.3 Identifiers (`IDENTIFIER`)

- User-defined names beginning with an ASCII letter or underscore (`[a-zA-Z_]`) followed by letters, digits, or underscores (`[a-zA-Z0-9_]*`).
- Every valid identifier occurrence is recorded in a shared `IdentifierTable` tracking 1-based coordinates and total frequency.

1.4 Operators and Delimiters

- **Mathematical Operators (`MATHOP`):** `+`, `-`, `*`, `/`, `%`
- **Logical and Relational Operators (`LOGOP`):** `==`, `!=`, `<`, `<=`, `>`, `>=`, `&&`, `||`, `!`
- **Assignment Operator (`ASSIGN`):** `=`
- **Delimiters:** `(`, `)`, `{`, `}`, `[`, `]`, `;`, `,`, `.`
- **Stream Termination:** An explicit `EOF` token is emitted at the end of every stream.

---

2. How Errors Were Handled

The scanner follows a non-halting error recovery strategy: when an error is detected, an error diagnostic is recorded with exact 1-based coordinates (`line`, `column`), and scanning continues to capture all remaining defects in a single pass.

Four required error categories are detected:

1. Invalid Characters (`LEX_INVALID_CHARACTER`): Characters not in the Mini Java alphabet (such as `@`, `#`, `$`) or lone bitwise operators (`&`, `|`) are reported with their coordinates. The invalid character is skipped and scanning resumes.

2. Invalid Identifiers (`LEX_INVALID_IDENTIFIER`): Identifiers prefixed with digits (e.g., `1name`, `2_count`, `12.34abc`). The entire alphanumeric sequence is consumed as a single unit to prevent cascaded parsing errors, and reported without emitting an identifier token.

3. Unclosed String Literals (`LEX_UNCLOSED_STRING`): Quoted literals that reach an unescaped newline or end-of-file without a closing quote. The error coordinate points directly to the opening quote.

4. Unclosed Multi-line Comments (`LEX_UNCLOSED_COMMENT`): Block comments (`/* ... */`) that reach end-of-file without a closing `*/`. The error coordinate points directly to the opening `/*`.

---

3. Challenges Faced and Solutions

3.1 Maximal Munch for Multi-Character Operators

- **Challenge:** Ambiguity between single-character and two-character operators (`=` vs `==`, `<` vs `<=`, `>` vs `>=`, `&` vs `&&`, `|` vs `||`).

- **Solution:** Maximal munch inspects the next character. If a two-character operator matches, both characters are consumed as `LOGOP`. Lone `&` and `|` are rejected as invalid characters since Mini Java does not support bitwise operators.

3.2 Distinguishing Floating-Point Dots from Delimiter Dots

- **Challenge:** The `.` character is both a member access delimiter (`System.out.println`) and a decimal separator in floats (`85.5`).
- **Solution:** A dot is treated as part of a `NUMBER` literal only when immediately preceded by digits and immediately followed by at least one digit. In all other cases, it is emitted as a `DOT` delimiter.

3.3 Digit-Prefixed Invalid Identifiers

- **Challenge:** Sequences like `1name` could prematurely split into a number (`1`) followed by an identifier (`name`), producing misleading syntax errors.
- **Solution:** After consuming digits, the scanner peeks ahead. If the subsequent character is an ASCII letter or underscore, the entire token is consumed as an invalid lexeme and a single `LEX_INVALID_IDENTIFIER` diagnostic is emitted.

3.4 Accurate Unclosed Multi-line Comment Tracking

- **Challenge:** When an unclosed block comment spans multiple lines, reporting the defect at EOF hides where the comment originally started.
- **Solution:** The scanner records the starting line and column upon encountering `/*`. If EOF is reached before `*/`, the diagnostic reports the exact opening position.